

12: Gaussian Processes

Date: February 24, 2026

We've seen how some kernels can be viewed as notions of similarity, with the corresponding models giving rise to smooth, k-NN like predictors e.g. with the RBF kernel. Additionally, we've (briefly) discussed how kernel matrices K with $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$ are therefore positive definite matrices whose entries describe the similarity between data points. **Gaussian processes** are a particular kind of kernelized regression algorithm that exploit the fact that kernel matrices K have many of the same properties as covariance matrices Σ in multivariate Gaussian distributions.

In some ways, Gaussian processes are among the simplest kernel methods we will study – if you understand multivariate Gaussians, you'll understand Gaussian processes.

1 Understanding multivariate Gaussians

The **multivariate Gaussian (or Normal) distribution** is the d dimensional extension of the univariate Gaussian. The probability density function of a Gaussian distribution looks like this:

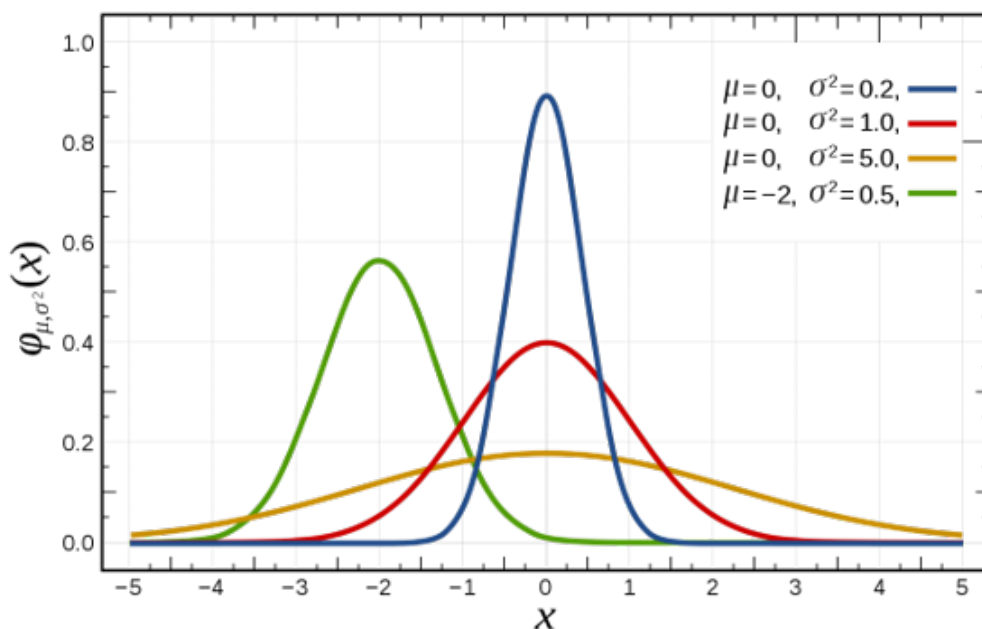


Figure 1: The univariate Gaussian pdf with various setting of μ and σ^2 .

Here, our Gaussian $\mathcal{N}(\mu, \sigma^2)$ is parameterized by a mean μ and variance σ^2 . σ^2 controls the spread of the distribution, μ controls the location of the distribution.

For multivariate Gaussians, we have a mean **vector** μ and **covariance matrix** Σ , $\mathcal{N}(\mu, \Sigma)$. When

we say:

$$\mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} \sim \mathcal{N} \left(\mu = \begin{bmatrix} \mu_1 \\ \vdots \\ \mu_n \end{bmatrix}, \Sigma = \begin{bmatrix} \sigma_{11}^2 & \cdots & \sigma_{1n} \\ \vdots & \ddots & \vdots \\ \sigma_{n1} & \cdots & \sigma_{nn}^2 \end{bmatrix} \right)$$

What we are specifying is a set of n Gaussians $y_i \sim \mathcal{N}(\mu_i, \sigma_{ii}^2)$, *plus how these Gaussians relate to each other*. Indeed, the **marginal** distribution of y_i for a multivariate Gaussian is just $\mathcal{N}(\mu_i, \sigma_{ii}^2)$. In other words, μ_i specifies the expected value of y_i , and σ_{ii}^2 specifies the variance of y_i . The off-diagonal **covariances** σ_{ij} in the covariance matrix specify how related y_i and y_j are. Indeed, the **correlation** between y_i and y_j is precisely:

$$\text{Corr}(y_i, y_j) = \frac{\text{Cov}(y_i, y_j)}{\sqrt{\text{Var}(y_i)\text{Var}(y_j)}} = \frac{\sigma_{ij}}{\sqrt{\sigma_{ii}^2 \sigma_{jj}^2}}$$

When $\sigma_{ij} = 0$ for all i and j , the multivariate Gaussian corresponds to just describing n independent univariate Gaussians: one for each y_i .

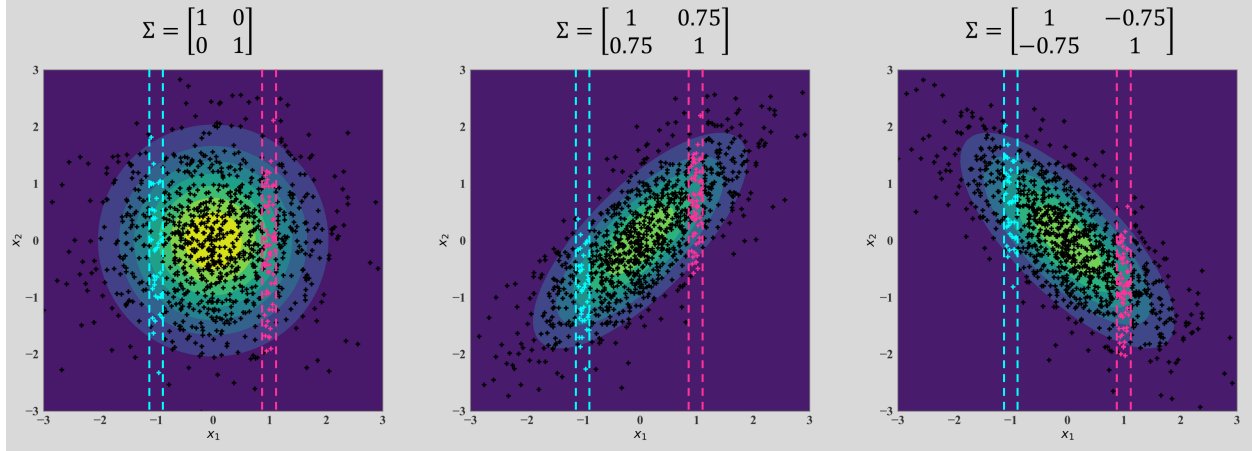


Figure 2: Multivariate Gaussian distributions over x_1, x_2 with $\mu = [0, 0]$ and various settings of the covariance matrix. Crosses are samples from the distribution, and the contour plot displays the pdf of the distribution. **Left:** The Gaussians are independent. Knowing that x_1 is somewhere in the narrow blue slice versus the narrow pink slice doesn't tell us anything about the value of x_2 . **Middle:** With positive correlation, knowing that x_1 takes a value of around 1 tells us we're more likely to find larger values of x_2 than if x_1 took values around -1 . **Right:** Negative correlation tells us the opposite – the larger x_1 gets, the smaller we expect x_2 to be.

The figure above depicts three example two dimensional Multivariate normal distributions. For each distribution, the mean is the all zeros vector and the diagonal elements of Σ are 1. With 0 covariance (and therefore 0 correlation), knowing that x_1 is somewhere in the narrow blue slice versus the narrow pink slice doesn't tell us anything about the value of x_2 . With positive correlation, knowing that x_1 takes a value of around 1 tells us we're more likely to find larger values of x_2 than if x_1 took values around -1 . Negative correlation tells us the opposite – the larger x_1 gets, the smaller we expect x_2 to be.

2 Functions as big multivariate Gaussians.

How can we use multivariate Gaussians to model the labels of data? Imagine I want to develop a model for the labels of a bunch of 1 dimensional data $x_1, \dots, x_n$ evenly spaced between 0 and 1. The most naive thing I could possibly do with Gaussians is to say that the labels $y_1, \dots, y_n$ are independent and identically distributed with the same mean and variance: $y_i \sim \mathcal{N}(0, 1)$. This is equivalent to saying that $\mathbf{y} \sim \mathcal{N}(0_d, I_d)$.

If I draw a sample from this model with say $n = 10$ evenly spaced points, I get something that looks like this:

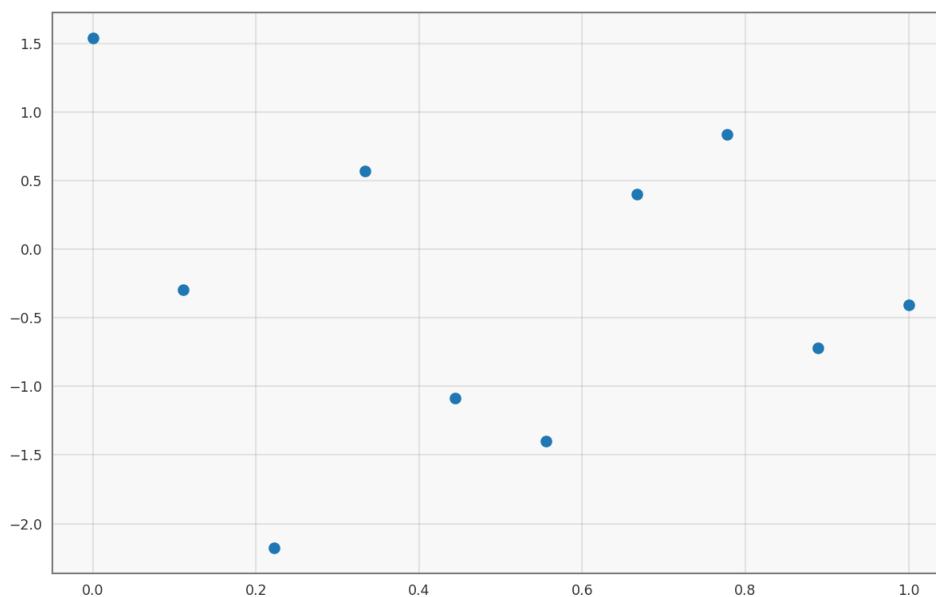


Figure 3: 10 evenly spaced points between 0 and 1 with labels drawn iid from $\mathcal{N}(0, 1)$.

This, of course, looks nothing like a function. Things get even worse if I have, say, 500 evenly spaced points and I again draw their labels iid:

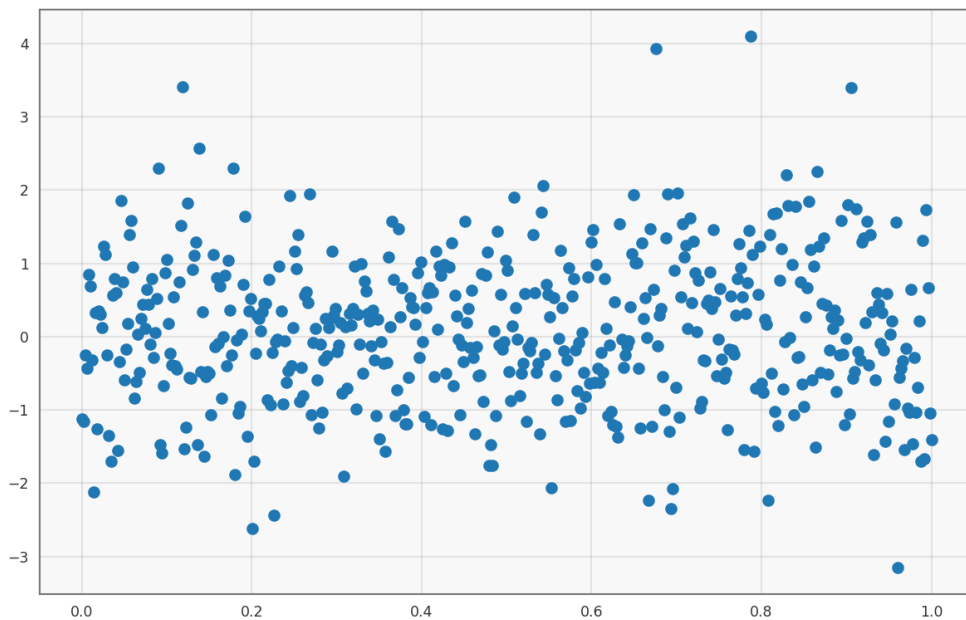


Figure 4: 500 evenly spaced points between 0 and 1 with labels drawn iid from $\mathcal{N}(0, 1)$.

What a mess! By *why* is this a mess, though? What makes this look nothing like a nice, smooth function? The reason is simple: the point at $x = 0$ and the immediate next point at $x = 0.0020$ have completely uncorrelated labels! What we need is, whenever we draw a sample of $y_1, \dots, y_n$, for the labels of points that are really close together to be *highly correlated*.

In other words, looking at the equation for correlation above, what we want is for σ_{ij} to become larger when $\mathbf{x}_i$ and $\mathbf{x}_j$ are close together, and for it to become small when they are far apart (because $x = 0$ doesn't necessarily need to have a correlated label with $x = 1$ all the way on the other end of the space). But wait, we've already seen a matrix that does that – the RBF kernel matrix!

What happens if we repeat the same procedure as the last two plots with evenly spaced data $\mathbf{x}_1, \dots, \mathbf{x}_n$, but instead of drawing the labels $\mathbf{y} \sim \mathcal{N}(0, I_d)$, we draw $\mathbf{y} \sim \mathcal{N}(0, K)$, where K is the $n \times n$ RBF kernel matrix with $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$? You get something like this:

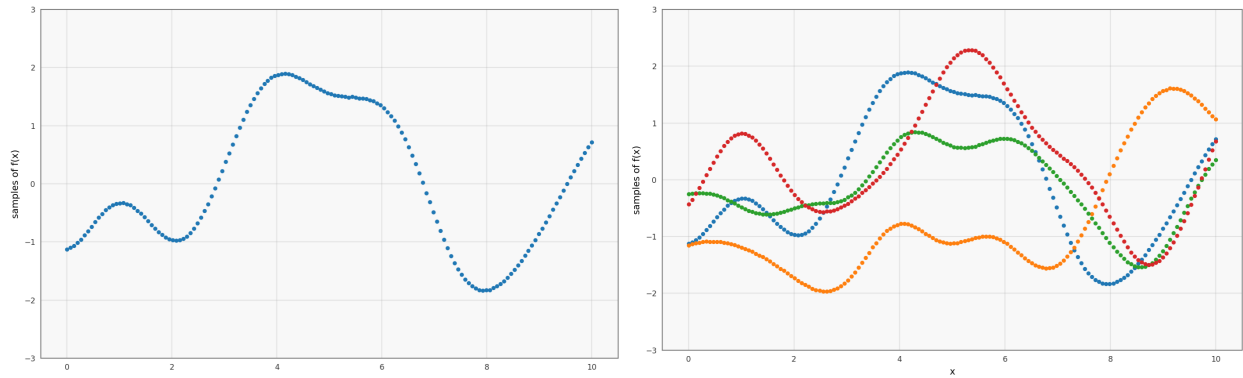


Figure 5: **Left:** One sample from $\mathcal{N}(0, K)$. **Right:** 4 samples from $\mathcal{N}(0, K)$.

Those look a lot more like functions! Intuitively, this makes sense: the RBF kernel is basically saying that if $\mathbf{x}_i$ and $\mathbf{x}_j$ are close together, samples of y_i and y_j should have high correlation. That’s exactly what our mental model of a “smooth” function is though, so it’s not surprising that samples from this kind of model look a lot like random smooth functions.

This simple sampling procedure gives us a great way to understand what kernels are telling us about what functions they model look like. For example, consider the following suggestively named *periodic kernel*:

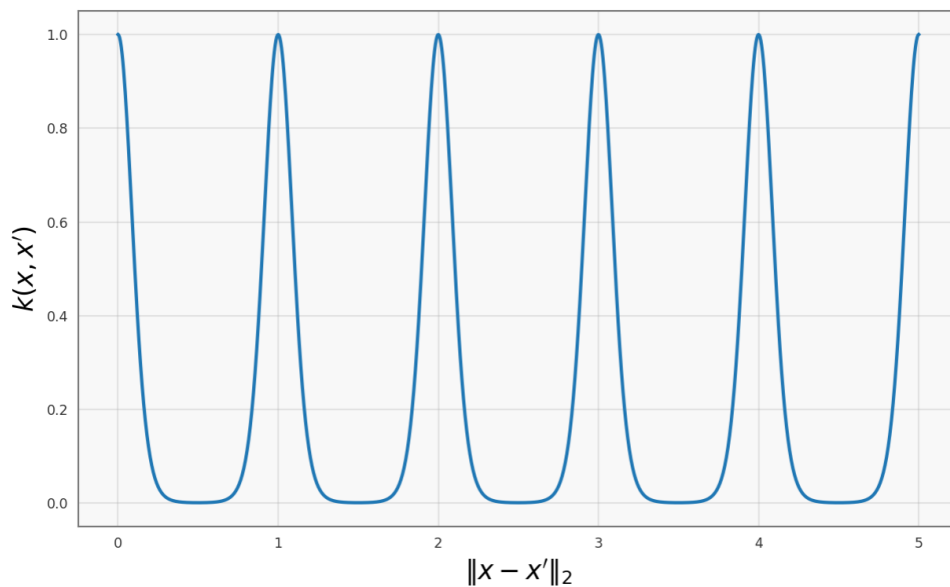


Figure 6: The periodic kernel.

Here, we see something that looks like an RBF kernel from a distance of 0 to about 0.5 or so, but then we see a perfect correlation / covariance of 1 again at a distance of 1. What should we expect samples from $\mathcal{N}(0, K)$ to look like if we use this kernel function to form K ? Well, points that are very close (e.g., distance close to 0) will still be very correlated. This time, however, points that

are at a distance of 1 will also be perfectly correlated, and a distance of 2, and so on. That’s what a periodic function looks like! Indeed, here are some samples of $\mathcal{N}(0, K)$:

3 Using multivariate Gaussians for learning.

In the previous section, what we saw is that samples from Gaussians with covariance matrices that are kernel matrices kind of “look” like nice functions. How do we turn this into a learning algorithm? The picture version of what we want to do is this:

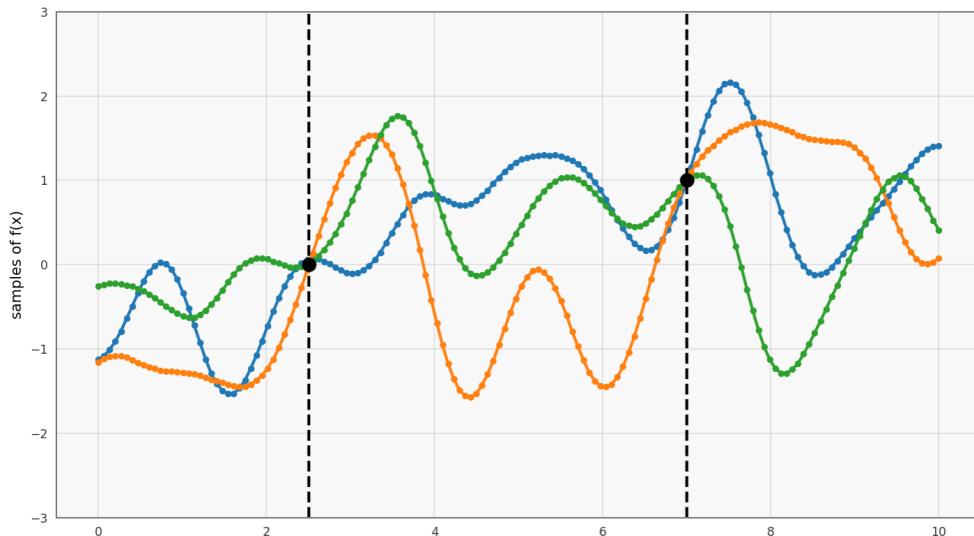


Figure 7: Samples from something like $\mathcal{N}(0, K)$, but where we’ve *conditioned* on two observations.

Here, we’re still drawing samples like before, but now we’ve *conditioned* on two observations – we *know* the function takes those particular values at those data points. In math, what we’re doing is exactly conditioning. Each of the dots on the plot represents a test point $\mathbf{z}_1, \dots, \mathbf{z}_T$ and there are T of them. Additionally, there are now two *training* points – the large black dots. The model we’re using basically says:

$$\begin{bmatrix} \mathbf{y}_x \\ \mathbf{y}_z \end{bmatrix} \sim \mathcal{N}\left(0, \begin{bmatrix} K_{xx} & K_{xz} \\ K_{xz}^\top & K_{zz} \end{bmatrix}\right)$$

Let’s define the pieces of notation:

1. $\mathbf{y}_x$ are the labels of the training data points. In our example, this is a length 2 vector.
2. $\mathbf{y}_z$ are the labels of the test points. In our example, there are 150 small dots on the plot, so this is a length 150 vector.
3. K_{xx} is the kernel matrix for just the training points, where $[K_{xx}]_{ij}$ is the kernel value between the i th and j th training point. In our example, this is a 2×2 matrix.
4. K_{zz} is the kernel matrix for just the test points, where $[K_{zz}]_{ij}$ is the kernel value between the i th and j th test point. In our example, this is a 150×150 matrix.

5. K_{xz} is the kernel matrix between the training points and test points, where $[K_{xz}]_{ij}$ is the kernel value between the i th training example and the j th test example. In our example, this is a 2×150 matrix.

The distribution we want is the probability distribution over the **labels of the test points, given the training labels**. In other words:

$$\text{Goal: } p(\mathbf{y}_z \mid \mathbf{y}_x)$$

This distribution is exactly what we want machine learning to do: we know the training data labels $\mathbf{y}_x$, and we want to compute some estimate—in this case a distribution—of the test labels $\mathbf{y}_z$ that we don't know! Fortunately, with Gaussians this is very easy: it turns out that $p(\mathbf{y}_z \mid \mathbf{y}_x)$ is Gaussian with closed form expressions for the new mean and covariance functions:

$$p(\mathbf{y}_z \mid \mathbf{y}_x) = \mathcal{N}(K_{zx}K_{xx}^{-1}\mathbf{y}_x, K_{zz} - K_{xz}^\top K_{xx}^{-1}K_{xz})$$

This is *almost* Gaussian process regression. The one last ingredient we need is that we might not have observed the labels of the training points exactly, but they might have been noisy (like in linear regression). The picture version of this is:

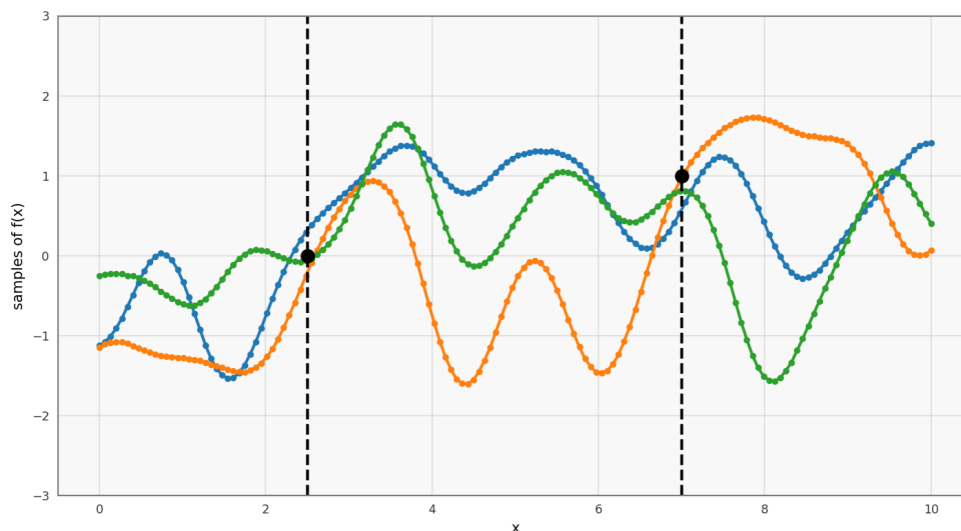


Figure 8: Samples from something *like* $\mathcal{N}(0, K)$, but where we've *conditioned* on two observations, but the observations were noisy.

Here, the functions no longer go *exactly* through the black circles, because we assume those labels were made with noise. The only change we need to make is to augment the model above slightly:

$$\begin{bmatrix} \mathbf{y}_x \\ \mathbf{y}_z \end{bmatrix} \sim \mathcal{N}\left(0, \begin{bmatrix} K_{xx} + \sigma_n^2 I & K_{xz} \\ K_{xz}^\top & K_{zz} \end{bmatrix}\right)$$

This augmentation corresponds exactly to saying that we observed the labels corrupted by Gaussian noise σ_n^2 . Why? Because the sum of Gaussians is Gaussians – if $\mathbf{y}_x$ has covariance K_{xx} , then $\mathbf{y}_x + \epsilon$ with $\epsilon \sim \mathcal{N}(0, \sigma_n^2 I)$ has covariance $K_{xx} + \sigma_n^2 I$.

The above defined a joint distribution over $p(\mathbf{y}_x, \mathbf{y}_z)$. However, what we really want is to make predictions for the test labels $\mathbf{y}_z$ given the training labels $\mathbf{y}_x$, or $p(\mathbf{y}_z|\mathbf{y}_x)$. Using a well-known fact from multivariate Gaussian distributions¹, this conditional distribution is another multivariate Gaussian with the following form:

$$p(\mathbf{y}_z | \mathbf{y}_x) = \mathcal{N}(K_{zx}(K_{xx} + \sigma_n^2 I)^{-1} \mathbf{y}_x, K_{zz} - K_{xz}^\top (K_{xx} + \sigma_n^2 I)^{-1} K_{xz})$$

And we're done. The above is Gaussian process regression. When plotting the above fit, we might plot something like this:

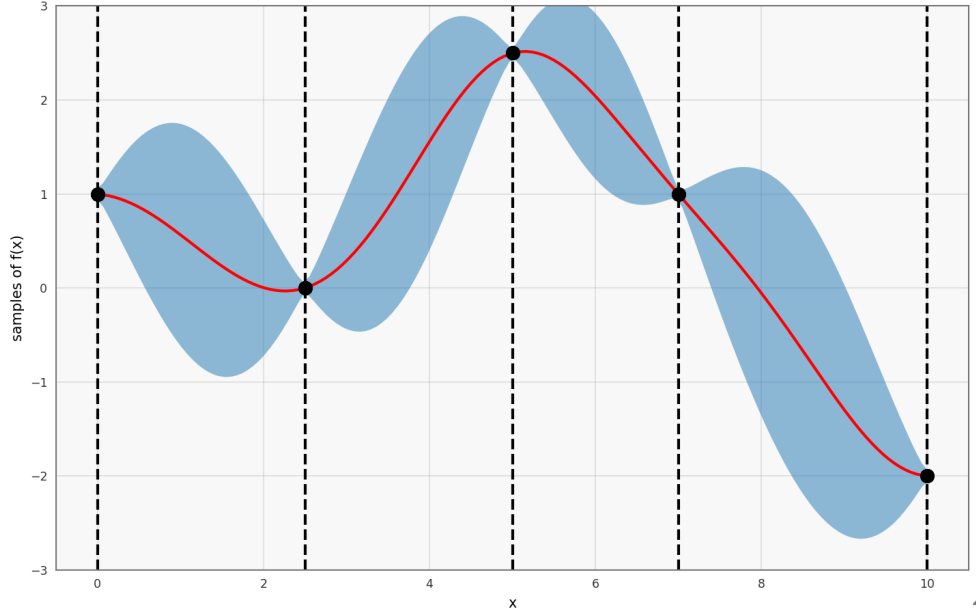


Figure 9: A GP regression model fit to 5 data points. The red curve is the mean of the distribution above where every point on the x axis is a test point. The blue shaded region is the variance at each test point (e.g., the diagonal of the covariance matrix)

4 Some final remarks.

First, let's compare the mean of the distribution above to kernel ridge regression. Recall that, in kernel ridge regression, the optimal $\boldsymbol{\alpha}^*$ is given by:

$$\boldsymbol{\alpha}^* = (K_{xx} + \lambda I)^{-1} \mathbf{y}_x$$

Here, we've used the notation we used above. K_{xx} is still the kernel matrix between all training examples, and $\mathbf{y}_x$ are the training labels. Furthermore, the final model is:

$$h(\mathbf{z}) = \sum_{i=1}^n \alpha_i k(\mathbf{x}_i, \mathbf{z}) = \mathbf{k}_{zx}^\top \boldsymbol{\alpha}$$

¹A proof for the conditional distribution of a multivariate Gaussian can be found at <https://statproofbook.github.io/P/mvn-cond.html>

Where $\mathbf{k}_{zx} = [k(\mathbf{z}, \mathbf{x}_1), \dots, k(\mathbf{z}, \mathbf{x}_n)]$. If we plug $\boldsymbol{\alpha}^*$ in for $\boldsymbol{\alpha}$, we get:

$$h(\mathbf{z}) = \mathbf{k}_{zx}^\top (K_{xx} + \lambda I)^{-1} \mathbf{y}_x$$

This is *exactly* the Gaussian process mean prediction, just with $\sigma_n^2 = \lambda$! Thus, GP regression and Kernel ridge regression have the same predictive mean. The GP regressor has two main advantages:

1. GP regression also gives you *uncertainty estimation*. In addition to a mean prediction, you also get a variance around each prediction, which can be very useful if you want to know when your model is unconfident.
2. While we won't discuss how to do this, there is a way to learn GP hyperparameters (σ_n^2 , the kernel lengthscale σ^2 , etc.) using gradient descent on a good objective.